

RETAIL INVENTORY MANAGEMENT SYSTEM

Our project, a Retail Inventory Management System, is designed to modernize inventory management for small retail businesses. It focuses on tracking stock levels, managing sales, and generating reports, aiming to enhance efficiency and reduce errors, making the lives of such business owners easier. This system leverages essential programming concepts to provide a user-friendly, well-organized, and speedy solution for retail operations.

BACKGROUND

Our research on small businesses and the challenges they face in everyday life, such as keeping track of inventory supervision, calculating sales, and general human error in record management, developed the idea for a retail management system. Throughout the investigation, one persistent issue was either the overly complex software or the outdated manual processes these business owners were using.

The selection of this project was further motivated by its practical implementation in the real world and its ability to simplify daily operations for business owners. By addressing these challenges, the system aims to provide an accessible and efficient solution that combines ease of use with reliability, ensuring that small businesses can focus more on growth rather than administrative tasks.

PROJECT SPECIFICATION

This system will handle inventory management, sales tracking, and report generation. The main target of this software is small business owners with minimal or no technical expertise.

Built exclusively using the C programming language, this project ensures compatibility with a wide range of IDEs.

PROBLEM ANALYSIS

The current methods used for business management are not prone to errors. Manual processes often lead to inaccuracies in record keeping, such as incorrect stock levels and calculation errors, which can cause major damage to the business. Furthermore, existing software solutions are often too complex or expensive, making them out of reach and impractical for small businesses, especially those with no major technical skills.

SOLUTION DESIGN

To tackle the problems defined above we have created a software with user-friendly interface to add, edit, and delete inventory records. We have also introduced functions that will automate sales calculations based on day-to-day sales in the store. Additionally, we have incorporated alerts to notify managers about low stock levels, ensuring timely restocking. The sales calculations update the owners about the most sold and least sold products, urging them to restock the most demanded items. Finally, we have added a feedback form to ask the customers for their comments on their shopping experience with the current inventory.

IMPLEMENTATION AND TESTING

Implementation

The Retail Inventory Management System was implemented using the C programming language, ensuring a lightweight, fast, and portable solution for small retail businesses. The development process followed a modular design, separating the system into key functional components:

1. **Inventory Management:** Handles adding, editing, deleting, and viewing inventory records.
2. **Sales Tracking:** Automates the recording of daily sales, calculating revenue, and maintaining sales history.
3. **Alert System:** Generates notifications for low stock levels based on predefined thresholds.
4. **Report Generation:** Creates summary reports highlighting key metrics such as total sales, most/least sold items, and inventory status.
5. **Feedback System:** Collects customer feedback to help business owners improve operations.

The implementation leveraged key programming techniques such as:

- **File Handling:** For persistent storage of inventory data and sales records.
- **Dynamic Memory Allocation:** Ensuring efficient use of memory for inventory operations.
- **Error Handling:** To validate user inputs and ensure smooth program execution.
- **Menu-Driven Interface:** Simplified navigation through clear options, designed with the user's technical limitations in mind.

The system was rigorously documented, including inline comments for maintainability and ease of future modifications.

Testing

To ensure the reliability and functionality of the system, extensive testing was carried out. The testing phase included unit testing where each module was tested independently to verify its behavior under various conditions, integration testing as the modules were combined, and interactions between them were validated to ensure seamless operation, and performance testing to ensure it performed efficiently even with a large amount of data.

PROJECT BREAKDOWN STRUCTURE

Program Overview

This program simulates a business management system that handles inventory, checkout, and customer feedback.

1. Main Function: `main()`
 - Displays Welcome Message:

Prints the greeting message to the user.

- Initial Setup Prompt:

Ask the user to press 1 to begin setting up the initial inventory.

If the user enters something other than 1, the program exits.

- Calls `InitialSetup()`:

Initiates the inventory setup process.

- Menu for Options:

Displays a menu asking the user to choose one of the following:

Set up inventory

Checkout

Feedback

Exit Program

Waits for user input and processes the selection using a switch statement:

- Option 1: `NewSetup()`

Deletes old files and sets up new inventory.

- Option 2: `Checkout()`

Handles the process of purchasing items from inventory.

- Option 3: `Feedback()`

Collects and processes customer feedback.

- Option 4: Exit

Exits the program.

Function: InitialSetup():

- **File Creation for Categories:**
 - Opens `FileNam` to store category file names.
 - Asks the user how many categories they want to create.
 - Iterates to collect category names and writes them into `FileName.txt`.
 - For each category, creates a new file with a `.txt` extension for inventory items.
- **Item Data Collection:**
 - Prompts the user to enter item details (name, quantity, price) for each category.
 - The user can add multiple items in each category or end by entering `1`.
- **End of Setup:**
 - Closes the category file after all items are added.
 - Moves to the next category if needed.

1. Function: DeleteOldFiles()

File Deletion:

Reads the `FileName.txt` file for existing category files.

Iterates through each category file name listed in `FileName.txt`.

Attempts to delete each file and prints a success or failure message.

1. Function: NewSetup()

Deletes Old Files:

Calls `DeleteOldFiles()` to remove existing category files.

Initial Setup:

Calls `InitialSetup()` to prompt the user to set up a new inventory.

1. Function: Checkout()

Item Search and Checkout Process:

Asks for the category of the item and checks if the category exists.

Prompts the user for the item name and desired quantity.

Opens the corresponding category file for reading.

Item Inventory Update:

Reads item data from the file and checks if the item exists.

If found, check if sufficient stock is available for purchase.

Updates stock and calculates the total cost if available.

If there is insufficient stock, it displays a message but keeps the original stock value.

Writes updated stock data back to a temporary file.

File Update:

After processing the purchase, the program deletes the original category file and renames the temporary file to replace it.

Purchase Confirmation:

Displays the purchase confirmation with the total cost if the purchase was successful.

1. Function: feedback()

Customer Feedback Collection:

Ask the user a series of questions regarding their shopping experience.

Prompts for feedback if any "no" answers are given.

Store Feedback Processing:

If the majority of responses are "yes", thank the user for their positive feedback.

If the majority are "no", ask for suggestions on how to improve and store that feedback.

End of Feedback:

Prints a message thanking the user and inviting them back.

File Handling and General Program Flow:

- Category Files:

Each category is represented by a .txt file storing items in the format: item_name = quantity @ price.

When an item is purchased, the inventory is updated accordingly in the file.

- Temporary Files:

A temporary file is used during the checkout process to write the updated inventory data, then replace the original category file.

Other Considerations

- User Input Validation:

The program assumes valid input for quantities and prices (does not handle invalid input or edge cases like negative values).

- Buffer Management:

The getchar() and fgetc() functions are used to clear input buffers where necessary to prevent issues with reading data.

RESULTS

The system demonstrated robustness in handling inventory and sales data without errors. Testers reported that the interface was intuitive and required minimal learning, and the generated reports provided valuable insights into store operations. Any identified bugs, such as misaligned report calculations or incorrect alert triggers, were resolved promptly. This comprehensive implementation and testing process ensured that the Retail Inventory Management System is a reliable, user-friendly tool for small business owners.

```

-----WELCOME TO YOUR BUSINESS MANAGEMENT PAGE-----
LETS FIRST START BY SETTING UP THE INITIAL INVENTORY
Enter 1 to continue: 1
How many categories do you want in your shop?: 2
Enter the category number 1: fruits
Enter the item name in category 1: bananas
Enter the amount of the item: 200
Enter the price for bananas: 50
Enter 1 to end adding items to this category or -1 to continue: -1
Enter the item name in category 1: apples
Enter the amount of the item: 300
Enter the price for apples: 75
Enter 1 to end adding items to this category or -1 to continue: 1
Enter the category number 2: vegetables
Enter the item name in category 2: onion
Enter the amount of the item: 600
Enter the price for onion: 150
Enter 1 to end adding items to this category or -1 to continue: -1
Enter the item name in category 2: carrots
Enter the amount of the item: 400
Enter the price for carrots: 120
Enter 1 to end adding items to this category or -1 to continue: 1

Which function do you want to choose:
1) Set up inventory
2) Checkout
3) Feedback
4) Exit Program
Enter your choice: 2
Enter the category of the item: fruits
Enter the name of the item: bananas
Enter the quantity you want to buy: 12
You have purchased 12 units of 'bananas'. Total cost: $600.00

Which function do you want to choose:
1) Set up inventory
2) Checkout
3) Feedback
4) Exit Program
Enter your choice: 2
Enter the category of the item: vegetables
Enter the name of the item: carrots
Enter the quantity you want to buy: 20
You have purchased 20 units of 'carrots'. Total cost: $2400.00

You have purchased 20 units of 'carrots'. Total cost: $2400.00

Which function do you want to choose:
1) Set up inventory
2) Checkout
3) Feedback
4) Exit Program
Enter your choice: 3
Did you find everything you were looking for? (yes/no): yes
Was everything okay with your shopping experience? (yes/no): no
Would you shop with us again in the future? (yes/no): yes
Would you recommend our store to others? (yes/no): no
Did you find our store easy to navigate? (yes/no): no

What would you like us to improve? (Type your suggestion): I would appericiate more assistance.

Thank you for letting us know. I will share your feedback with our team so we can improve.
Have a great day and come back anytime.

Which function do you want to choose:
1) Set up inventory
2) Checkout
3) Feedback
4) Exit Program
Enter your choice: 4
Exiting program...
-----
Process exited after 170.9 seconds with return value 0

```

CONCLUSION

The Retail Inventory Management System solves common problems small business owners face in managing inventory and sales. It provides a simple, efficient, and affordable solution that replaces outdated manual processes and complicated software. With its easy-to-use design and careful testing, the system is reliable and accessible, even for those with little technical knowledge. This project shows how programming can be used to solve real-life problems and can be improved further in the future.